You can view this report online at : https://www.hackerrank.com/x/tests/1797042/candidates/59853390/report

| | |
|---|---|
| Full Name: | Ben Skiff |
| Email: | bskiff@sourceallies.com |
| Test Name: | **Tech Trials: Auth & Analysis** |
| Taken On: | 18 Jan 2024 19:39:52 CST |
| Time Taken: | 27 min 18 sec/ 60 min |
| Invited by: | Carter |
| Invited on: | 18 Jan 2024 13:04:03 CST |
| Skills Score: | Problem Solving (Basic) 50/100<br>Problem Solving (Intermediate) 75/150 |
| Tags Score: | Arrays 75/75<br>Easy 50/100<br>Heaps 0/75<br>Interviewer Guidelines 75/75<br>Medium 75/150<br>Real-World 125/250<br>Strings 75/75 |

**50%**

**125/250**

scored in **Tech Trials: Auth & Analysis** in 27 min 18 sec on 18 Jan 2024 19:39:52 CST

Recruiter/Team Comments:

*No Comments.*

| | Question Description | Time Taken | Score | Status |
|---|---|---|---|---|
| Q1 | Error Log Extraction > Coding | 12 min 2 sec | 50/ 50 | ✓ |
| Q2 | Request Parser > Coding | 44 sec | 0/ 50 | ✗ |
| Q3 | Glide Typing > Coding | 13 min 51 sec | 75/ 75 | ✓ |
| Q4 | Authentication System > Coding | 21 sec | 0/ 75 | ✗ |

QUESTION 1

Correct Answer

Score 50

## Error Log Extraction > Coding

Easy | Real-World

QUESTION DESCRIPTION

Develop a function that processes a stream of log data from AWS CloudWatch, represented as a 2D array of strings called `logs`. Each inner list `logs` contains log information, including the year, date, status, and message.

The function should return all logs with ERROR or CRITICAL status sorted by their time of arrival and if two logs arrive at the same time return them in the order which appears first in `logs`. The log date is in DD-MM-YYYY format, and the log time is in HH:MM format.

**Example**

Suppose *logs* = [["01-01-2023", "14:00", "ERROR", "failed"], ["01-01-2023", "15:00", "INFO", "established"], ["01-01-2023", "01:30", "ERROR", "failed"]].

| Log Index | Date | Time | Status | Message |
|---|---|---|---|---|
| 1 | 01-01-2023 | 01:30 | ERROR | failed |
| 2 | 01-01-2023 | 14:00 | ERROR | failed |
| 3 | 01-01-2023 | 15:00 | INFO | established |

The answer is [["01-01-2023", "01:30", "ERROR", "failed"], ["01-01-2023", "14:00", "ERROR", "failed"]]. This is because the input contains two log entries with ERROR status, and they are sorted in ascending order of their time of arrival.

**Function Description**

Complete the function *extractErrorLogs* in the editor below.

*extractErrorLogs* has the following parameter(s):

*string logs[n][4]:* the unfiltered, unsorted log information

**Returns**

*string[][4]:* the filtered error log

**Constraints**

- $1 \le n$ (size of *logs*) $\le 10^5$
- 01 ≤ DD (day) ≤ 31
- 01 ≤ MM (month) ≤ 12
- 2000 ≤ YYYY (year) ≤ 3000
- 00 ≤ HH (hour) ≤ 23
- 00 ≤ MM (minutes) ≤ 59

**▼ Input Format For Custom Testing**

The first line contains an integer, *n*, the number of elements in *logs*.

The next line contains an integer, *4*, the number of columns in *logs*.

Each line *i* of the *n* subsequent lines (where $0 \le i < n$) contains 4 strings describing *logs[i]*.

**▼ Sample Case 0**

**Sample Input For Custom Testing**

```
STDIN                                          FUNCTION
-----                                          --------
3                                    →         logs[] size n = 3
4                                    →         logs[][] size const = 4
01-01-2023 15:00 ERROR failed        →         logs = [["01-01-2023",
"15:00", "ERROR", "failed"], ["01-01-2023", "16:00", "SUCCESS",
"established"],
01-01-2023 16:00 SUCCESS established                  ["01-01-2022",
"18:00", "CRITICAL", "failed"]]
01-01-2022 18:00 CRITICAL failed
```

**Sample Output**

```
01-01-2022 18:00 CRITICAL failed
01-01-2023 15:00 ERROR failed
```

**Explanation**

*logs*[0] and *logs*[2] contain ERROR or CRITICAL status, and *logs*[2] has the earlier time of arrival

**▼ Sample Case 1**

**Sample Input For Custom Testing**

```
STDIN                                          FUNCTION
-----                                          --------
3                                       →      logs[] size n = 3
4                                       →      logs[][] size const = 4
05-08-2023 12:00 SUCCESS established    →      logs = [["05-08-2023",
"12:00", "SUCCESS", "established"], ["03-07-2023", "16:00", "SUCCESS",
"established"],
03-07-2023 16:00 SUCCESS established                ["04-12-2022",
"18:00", "SUCCESS", "established"]]
04-12-2022 18:00 SUCCESS established
```

**Sample Output**

**Explanation**

There are no logs with ERROR or CRITICAL STATUS, hence the answer is [].

INTERVIEWER GUIDELINES

**▼ Solution**

**Skills:** Sorting, Strings

We loop through each log and converts the `log_date` from DD-MM-YYYY format to YYYY-MM-DD format. This helps in sorting properly.
If the log has an ERROR or CRITICAL status, it is appended to the `ans` list.

The solution then sorts the `ans` list by the log arrival time and converts the date back to DD-MM-YYYY format.
Note: We make a tuple to help in sorting. The first element is the date, the second is the timestamp and the third is the index in the logs array.

Finally, the function converts the date back to original format and returns the `ans` list.

**Optimal Solution:**

```
def extractErrorLogs(logs):
    n = len(logs)

    ans = []

    for i in range(n):
        dd, mm, yy = logs[i][0].split('-')
        hh, mn = logs[i][1].split(':')
        logs[i][0] = f"{yy}-{mm}-{dd}"

        if logs[i][2] == "CRITICAL" or logs[i][2] == "ERROR":
            ans.append([logs[i][0], logs[i][1], i, logs[i][2], logs[i]
[3]])

    ans.sort()
```

```
    for i in range(len(ans)):
        ans[i] = [ans[i][0],ans[i][1], ans[i][3],ans[i][4]]
        yy, mm, dd = ans[i][0].split('-')
        ans[i][0] = f"{dd}-{mm}-{yy}"

    return ans
```

**▼ Complexity Analysis**

**Time Complexity** - O(nlogn)
This is because the function sorts the logs based on their time of arrival, which takes O(nlogn) time. The other operations in the function, such as the loop to convert the log date format and the loop to convert the date back to the original format, have a time complexity of O(n).

**Space Complexity** - O(n)
This is because the function creates a new list `ans` to store the logs that have ERROR or CRITICAL status, which could potentially contain all the logs in the input list.

**CANDIDATE ANSWER**

Language used: **TypeScript**

```typescript

/*
 * Complete the 'extractErrorLogs' function below.
 *
 * The function is expected to return a 2D_STRING_ARRAY.
 * The function accepts 2D_STRING_ARRAY logs as parameter.
 */

const logToEpochMs = (log: string[]): number => {
    const date = log[0];
    const time = log[1];
    // 01-01-2023
    const dateParts = date.split('-');
    // 2015-01-16T22:15:00
    const dateIsoFormat =
`${dateParts[2]}-${dateParts[1]}-${dateParts[0]}T${time}:00`;
    return Date.parse(dateIsoFormat);
};

function extractErrorLogs(logs: string[][]): string[][] {
    // Write your code here
    return logs.filter((log) => {
        const level = log[2];
        return level === 'ERROR' || level === 'CRITICAL';
    }).sort((a, b) => {
        return logToEpochMs(a) - logToEpochMs(b);
    });
}
```

| TESTCASE | DIFFICULTY | TYPE | STATUS | SCORE | TIME TAKEN | MEMORY USED |
|---|---|---|---|---|---|---|
| TestCase 0 | Easy | Sample case | Success | 1 | 0.0367 sec | 42 KB |
| TestCase 1 | Easy | Sample case | Success | 1 | 0.0493 sec | 41.8 KB |
| TestCase 2 | Easy | Sample case | Success | 1 | 0.0408 sec | 42.2 KB |
| TestCase 3 | Easy | Hidden case | Success | 2 | 0.0476 sec | 41.5 KB |

| | | | | | | |
|---|---|---|---|---|---|---|
| TestCase 4 | Medium | Hidden case | Success | 3 | 0.0834 sec | 50.3 KB |
| TestCase 5 | Medium | Hidden case | Success | 3 | 0.1213 sec | 49.7 KB |
| TestCase 6 | Medium | Hidden case | Success | 3 | 0.07 sec | 49.8 KB |
| TestCase 7 | Medium | Hidden case | Success | 3 | 0.0831 sec | 50.4 KB |
| TestCase 8 | Medium | Hidden case | Success | 3 | 0.067 sec | 50.1 KB |
| TestCase 9 | Hard | Hidden case | Success | 5 | 1.6639 sec | 101 KB |
| TestCase 10 | Hard | Hidden case | Success | 5 | 1.6202 sec | 100 KB |
| TestCase 11 | Hard | Hidden case | Success | 5 | 1.669 sec | 103 KB |
| TestCase 12 | Hard | Hidden case | Success | 5 | 1.3978 sec | 99.8 KB |
| TestCase 13 | Hard | Hidden case | Success | 5 | 1.5459 sec | 98.5 KB |
| TestCase 14 | Hard | Hidden case | Success | 5 | 2.3613 sec | 100 KB |

No Comments

**QUESTION 2**

Wrong Answer

Score 0

## Request Parser › Coding

Easy | Real-World

QUESTION DESCRIPTION

The process of initiating an action on a server is done through HTTP requests which are messages sent by the client. The two most commonly used HTTP requests are GET and POST. This task involves validating requests and parsing URL parameters as a comma-separated string. Authentication tokens for both GET and POST requests are sent as a URL parameter named "token". For validation of authentication, the tokens must be in a set of valid authentication tokens. In the case of a POST request, a CSRF (cross-site request forgery) token must also be provided. A POST request is considered valid if its authentication token is valid and its CSRF token is an alphanumeric value consisting only of lowercase letters and/or numbers with a minimum length of 8. Once a request is validated, the URL parameters must be parsed as a comma-separated string.

URL parameters are identified by the portion of the URL that comes after a question mark (?). They consist of a key and a value, separated by an equal sign (=). Multiple parameters are separated by an ampersand (&).

Implementation a request parser prototype. Given an array of strings, *valid_auth_tokens*, representing the valid authentication tokens, and a 2D array of strings, *requests*, representing the request types and URLs, for each request, return the request status ("VALID" or "INVALID"). If VALID, include a comma-separated string of parameters, i.e. "VALID,<param1_key>, <param1_value>,<param2_key>, <param2_value>".

**Example**

*valid_auth_tokens* = ["ah37j2ha483u", "safh34ywb0p5", "ba34wyi8t902"]
*requests* = [["GET", "https://example.com/?token=347sd6yk8iu2&name=alex"], ["GET", "https://example.com/?token=safh34ywb0p5&name=sam"], ["POST", "https://example.com/?token=safh34ywb0p5&name=alex"], ["POST", "https://example.com/?token=safh34ywb0p5&csrf=ak2sh32dy&name=chris"]].

- In the first request, the auth_token = 347sd6yk8iu2, which is not in the list of given tokens, so the request is INVALID. The string to be returned is "INVALID".
- In the second request, the auth_token = safh34ywb0p5, which is in the list of valid tokens, so the request is VALID. The request parameters are - name = sam. The string to be returned is "VALID,name,sam".
- In the third request, the auth_token = safh34ywb0p5, which is in the list of valid tokens, but since the request is a POST request, it must have a valid CSRF token. Since the given request doesn't have a CSRF token, the request is INVALID. The string to be returned is "INVALID".

- In the fourth request, the auth_token = safh34ywb0p5, which is in the list of valid tokens, but since the request is a POST request, it must have a valid CSRF token. CSRF_token = ak2sh32dy. It is an alphanumeric string with length 9 (>=8), so the given request is VALID. The request parameters are - name = chris. The string to be returned is "VALID,name,chris".

The responses returned are ["INVALID", "VALID,name,sam", "INVALID", "VALID,name,chris"].

**Function Description**

Complete the function *getResponses* in the editor below.

*getResponses* has the following parameter(s):

*string valid_auth_tokens[m]:* The list of valid authentication tokens the requests

*string requests[n][2]:* The list of requests made, their types and the associated URLs.

**Returns**

*string[n]:* the responses for each request

**Constraints**

- $1 \le m \le 2*10^3$
- $1 \le n \le 5*10^3$
- size(*valid_auth_token[i]*) = 12
- size(*requests[i][1]*) ≤ 200

**▼ Input Format For Custom Testing**

The first line contains an integer, *m*, the number of elements in *valid_auth_tokens.*

Each of the next *m* lines contains a string, *valid_auth_tokens[i]*

The next line contains an integer, *n*, the number of requests.

The next line contains a constant integer, 2.

Each of the next *n* lines contains two space-separated strings that represent *requests[i].*

**▼ Sample Case 0**

**Sample Input For Custom Testing**

```
STDIN                                                                FUNCTION
-----                                                                    --------
3
    →    m = 3
et51u8i9p1q7
    →    valid_auth_tokens = ["et51u8i9p1q7",
b8nn5j4om76v
                              "b8nn5j4om76v",
r5b019lmlp09
                              "r5b019lmlp09"]
2
    →    n = 2
2
    →    constant integer = 2
GET https://example.com/?
token=et51u8i9p1q7&id=2e3rt&name=alex                →    requests =
[["GET", "https://example.com/?token=et51u8i9p1q7&id=2e3rt&name=alex"],
POST https://example.com/?
token=r5b019lmlp09&csrf=ia+09&id=u78we&name=evan                    ["POST
", "https://example.com/?
token=r5b019lmlp09&csrf=ia+09&id=u78we&name=evan"]]
```

**Sample Output**

```
VALID,id,2e3rt,name,alex
INVALID
```

**Explanation**

- *auth_token* = et51u8i9p1q7, is in the list of valid tokens, so the given request is VALID. The request parameters are - id = 2e3rt, name = alex.
- *auth_token* = r5b019lmlp09, in the list of valid tokens, but since the request is a POST request, the CSRF token must be validated. The CSRF token, contains a non-alphanumeric character ("+"), and its length = 5(<8) so the POST request is INVALID.

## ▼ Sample Case 1

**Sample Input For Custom Testing**

```
STDIN
                                                                FUNCTION
-----
                                                                    --------
4
      →   m = 4
ggu814h5py8k
      →   valid_auth_tokens = ["ggu814h5py8k",
5lfa89797zi9
                               "5lfa89797zi9",
u63339410gj8
                               "u63339410gj8",
5hu1tnd901a4
                               "5hu1tnd901a4"]
3
      →   n = 3
2
      →   constant integer = 2
POST https://example.com/?
token=pz9iy1j438ol&csrf=a79i49&id=c7v35              →   requests =
[["POST", "https://example.com/?token=pz9iy1j438ol&csrf=a79i49&id=c7v35"],
POST https://example.com/?
token=ggu814h5py8k&csrf=79jm98h6&id=1856a
→              ["POST", "https://example.com/?
token=ggu814h5py8k&csrf=79jm98h6&id=1856a],
POST https://example.com/?
token=5lfa89797zi9&csrf=0y9v62o5&id=715w7&name=sam    →              ["PO
ST", https://example.com/?
token=5lfa89797zi9&csrf=0y9v62o5&id=715w7&name=sam"]]
```

**Sample Output**

```
INVALID
VALID,id,1856a
VALID,id,715w7,name,sam
```

**Explanation**

- *auth_token* = pz9iy1j438ol, not in the list of valid tokens, so the given request is INVALID.
- *auth_token* = ggu814h5py8k, in the list of valid tokens, but since the request is a POST request, the CSRF token must be validated. The CSRF *token* = 79jm98h6, consists of only alphanumeric characters, and its length = 8 so the given POST request is VALID. The request parameters are - id = 1856a, String to be returned = "VALID, id,1856a".
- *auth_token* = 5lfa89797zi9, in the list of valid tokens, but since the request is a POST request, the CSRF token is to be validated. CSRF token = 0y9v62o5, it consists of only alphanumeric characters and length = 8 so the given POST request is VALID. The request parameters are - id = 715w7, name = sam. String to be returned = "VALID, id,715w7, name, sam".

INTERVIEWER GUIDELINES

## ▼ Solution

**Skills:** e.g., Loops and counters, I/O, dynamic programming

The solution is implemented using a function named `parse_requests` that takes in three parameters - `valid_auth_tokens`, `requests`, and `n`. The first parameter `valid_auth_tokens` is a list of valid authentication tokens, the second parameter `requests` is a 2D list of strings that represents the requests, and the third parameter `n` represents the number of requests.

First, the function initializes an empty list named `results` that will hold the final results for each request. Then, for each request in the `requests` list, the function performs the following steps:

It splits the request into the request type and the URL using the `split` method and stores them in `req_type` and `url` variables respectively.

It extracts the authentication token from the URL using the `get_parameter` function and stores it in the `auth_token` variable.

It checks if the `auth_token` is in the list of `valid_auth_tokens`. If it is not, the function appends the string "INVALID" to the `results` list and moves on to the next request.

If the `auth_token` is valid, the function checks if the request type is "POST". If it is, the function extracts the CSRF token from the URL using the `get_parameter` function and stores it in the `csrf_token` variable.

It then checks if the `csrf_token` is an alphanumeric string of length at least 8. If it is not, the function appends the string "INVALID" to the `results` list and moves on to the next request.

If both the authentication token and the CSRF token are valid (in the case of a POST request), the function extracts the URL parameters using the `get_params` function and stores them in the `params` dictionary.

Finally, the function appends the string "VALID" to the `results` list followed by the comma-separated key-value pairs from the `params` dictionary. If the `params` dictionary is empty (i.e., there are no URL parameters), the function appends an empty string to the `results` list.

Once all the requests have been processed, the function returns the `results` list.

The `get_parameter` function takes in a URL string and a parameter name and returns the value of the parameter. It does this by first finding the index of the parameter name using the `find` method and then extracting the value using slicing.

The `get_params` function takes in a URL string and returns a dictionary of URL parameters. It does this by first finding the index of the question mark (?) using the `find` method and then extracting the substring after the question mark using slicing. It then splits this substring into a list of key-value pairs using the `split` method and stores them in a dictionary.

**Optimal Solution:**

```
def getResponses(valid_auth_tokens, requests):
    # Store the length of the valid_auth_tokens and requests
    num_valid_tokens = len(valid_auth_tokens)
    num_requests = len(requests)

    # Convert the valid_auth_tokens list to a set for faster lookups
    valid_tokens_set = set(valid_auth_tokens)

    # Initialize an empty list to store the responses
    responses = []

    # Loop through each request and check if it's valid
    for request in requests:
        # Extract the HTTP method (GET or POST) and URL string from the
request
        http_method, url = request[0], request[1]

        # Initialize the response to "VALID"
        response = "VALID"

        # Extract the token and CSRF value from the URL parameters
        token = ""
        csrf = ""
        url_parameters = url.split('?')[1].split('&')
        for parameter in url_parameters:
```

```
            key, value = parameter.split('=')
            if key == "token":
                token = value
                continue
            elif key == "csrf":
                csrf = value
                continue
            response += f",{key},{value}"  # Append the URL parameter to
the response string

        # Check if the token is valid
        if token not in valid_tokens_set:
            response = "INVALID"

        # If the request is a POST request, check if the CSRF value is at
least 8 characters long
        # and contains only lowercase letters and/or numbers
        if http_method == "POST":
            # likely solution
            if len(csrf) < 8 or not csrf.isalnum() or not csrf ==
csrf.lower():
            # alternative regex
            # if not re.match('^([a-zA-Z0-9]){8,}$',csrf):
                response = "INVALID"


        # Append the response ("VALID" or "INVALID") to the list of
responses
        responses.append(response)

    # Return the list of responses
    return responses
```

**▼ Complexity Analysis**

**Time Complexity** - O(n)

We iterate over the requests that takes O(n) time.

**Space Complexity** - O(n)

The responses array takes O(n) extra space.

**CANDIDATE ANSWER**

Language used: **TypeScript**

```
/*
 * Complete the 'getResponses' function below.
 *
 * The function is expected to return a STRING_ARRAY.
 * The function accepts following parameters:
 *  1. STRING_ARRAY valid_auth_tokens
 *  2. 2D_STRING_ARRAY requests
 */

function getResponses(valid_auth_tokens: string[], requests: string[][]):
string[] {

}
```

**Result:** Compilation Failed

Compile Message

```
Solution.ts(37,75): error TS2355: A function whose
declared type is neither 'undefined', 'void', nor 'any' must return a
value.
```

No Comments

QUESTION 3

Correct Answer

Score 75

## Glide Typing › Coding

Arrays | Medium | Strings | Real-World | Interviewer Guidelines

QUESTION DESCRIPTION

In this task, a basic prototype service for detecting the word typed by a user by swiping the finger from key to key on the keyboard, more popularly known as glide typing, is to be implemented. A user usually intends to type a subsequence of the characters they swiped over.

Given an input string, *s,* and an array of *n* strings, *dictionary,* find the lexicographically smallest word in the dictionary that is a subsequence of the given string *s.* If there is no such string, return the string "-1" as the answer.

Note: A string *x* is considered lexicographically smaller than another string *y* if the string *x* will occur before *y* in dictionary order.

**Example**

Suppose *s* = "hgferyjkllkop", and *dictionary* = [“coffee”, “coding”, “happy”, “hello”, "hop"]

The only possible valid words that are a subsequence of the given string are "hello" and "hop". Since "hello" is lexicographically smaller, report "hello" as the answer.

**Function Description**

Complete the function *getValidWord* in the editor below.

*getValidWord* takes the following arguments:

*string s:* the string given by the user

*string dictionary[n]:* the list of valid words

**Returns**

*string:* the lexicographically smallest word that is a subsequence of the user-given string

**Constraints**

- $1 \le |s| \le 20$
- $1 \le n \le 10^5$
- $1 \le |word[i]| \le 20$
- It is guaranteed that all the strings consists of lowercase English letters only.

**▼ Input Format For Custom Testing**

The first line contains a string, *s*.

The next line contains an integer, *n*, the number of elements in *dictionary*.

Each of the next n lines contains a string, *dictionary[i]*.

**▼ Sample Case 0**

**Sample Input For Custom Testing**

```
STDIN           FUNCTION
-----           --------
ocbms     →     s = "ocbms"
5         →     dictionary[] size n = 5
rdnothix →     dictionary = ["rdnothix", "obms", "qhlrpiaiv", "ms",
"jaupx"]
obms
qhlrpiaiv
ms
jaupx
```

**Sample Output**

```
ms
```

**Explanation**

The valid subsequence words are "ms" and "obms" out of which "ms" is smaller.

**▼ Sample Case 1**

**Sample Input For Custom Testing**

```
STDIN           FUNCTION
-----           --------
zafqz     →     s = "zafqz"
3         →     dictionary[] size n = 3
dvuyaufe →     dictionary = ["dvuyaufe", "gtxbc", "hbckuhyh"]
gtxbc
hbckuhyh
```

**Sample Output**

```
-1
```

**Explanation**

None of the words is a subsequence of the given string.

INTERVIEWER GUIDELINES

**▼ Solution**

**Skills:** Strings, Greedy Algorithms

For each word, we iterate over the string and maintain an index *i* which is incremented every time we find some character *x* in the string is the same as *word[i].* If *i* becomes equal to the length of the *word,* then this is a valid word and we maintain answer to find the minimum possible valid word.

**Optimal Solution:**

```
def getValidWord(s, dictionary):
    ans = "-1"
    for word in dictionary:
        i = 0
        for x in s:
            if x == word[i]:
                i += 1
            if i == len(word):
                ans = word if ans == "-1" else min(ans, word)
                break
    return ans
```

**▼ Complexity Analysis**

**Time Complexity** - O(n)

Assuming the length of the strings is small, we iterate over the list of words that takes O(n) time.

**Space Complexity** - O(1)

A constant amount of extra space is consumed by the solution.

**CANDIDATE ANSWER**

Language used: **TypeScript**

```typescript

/*
 * Complete the 'getValidWord' function below.
 *
 * The function is expected to return a STRING.
 * The function accepts following parameters:
 *  1. STRING s
 *  2. STRING_ARRAY dictionary
 */
const isValidWord = (validWord: string, s: string): boolean => {
    let remainingStringToSearch = s;

    for (let charIndex = 0; charIndex < validWord.length; charIndex++) {
        const charToCheck = validWord[charIndex];
        const index = remainingStringToSearch.indexOf(charToCheck);
        // The next character in the valid word wasn't contained in the
remaining string that we're search through
        if (index === -1) {
            return false;
        }
        remainingStringToSearch = remainingStringToSearch.slice(index);
    }
    return true;
}

function getValidWord(s: string, dictionary: string[]): string {
    const sortedDictionary = dictionary.sort();

    const validWord = sortedDictionary.find((validWord) =>
isValidWord(validWord, s));
    if (validWord) {
        return validWord;
    }
    return '-1';
}
```

| TESTCASE | DIFFICULTY | TYPE | STATUS | SCORE | TIME TAKEN | MEMORY USED |
|---|---|---|---|---|---|---|
| Testcase 0 | Easy | Sample case | Success | 1 | 0.0451 sec | 41.6 KB |
| Testcase 1 | Easy | Sample case | Success | 1 | 0.0504 sec | 41.9 KB |
| Testcase 2 | Easy | Sample case | Success | 1 | 0.0419 sec | 41.6 KB |
| Testcase 3 | Medium | Hidden case | Success | 4 | 0.0427 sec | 41.9 KB |
| Testcase 4 | Medium | Hidden case | Success | 4 | 0.0506 sec | 41.4 KB |
| Testcase 5 | Medium | Hidden case | Success | 4 | 0.0418 sec | 42 KB |
| Testcase 6 | Medium | Hidden case | Success | 4 | 0.0423 sec | 42 KB |
| Testcase 7 | Medium | Hidden case | Success | 4 | 0.0387 sec | 41.9 KB |
| Testcase 8 | Medium | Hidden case | Success | 4 | 0.0478 sec | 41.6 KB |
| Testcase 9 | Hard | Hidden case | Success | 8 | 0.138 sec | 55.6 KB |
| Testcase 10 | Hard | Hidden case | Success | 8 | 0.1534 sec | 57.6 KB |

| | | | | | | |
|---|---|---|---|---|---|---|
| Testcase 11 | Hard | Hidden case | Success | 8 | 0.225 sec | 58.3 KB |
| Testcase 12 | Hard | Hidden case | Success | 8 | 0.2156 sec | 61.7 KB |
| Testcase 13 | Hard | Hidden case | Success | 8 | 0.2563 sec | 59 KB |
| Testcase 14 | Hard | Hidden case | Success | 8 | 0.2518 sec | 60.3 KB |

No Comments

QUESTION 4

Wrong Answer

Score 0

## Authentication System › Coding

Medium | Heaps | Real-World

QUESTION DESCRIPTION

Develop a session-based authentication system where each user session generates a new token with a specified time-to-live (TTL) expiration. The TTL expiration is calculated as the current time plus the time-to-live seconds. If a token is renewed, the TTL is extended to the time-to-live seconds after the time of the renewal.

There are three types of queries in this system:

- "generate <token_id> <current_time>": Creates a new token with the given *token_id* and *current_time*.
- "renew <token_id> <current_time>": Renews an unexpired token with the given *token_id* and *current_time*. If there is no unexpired token with the specified *token_id*, the request is ignored.
- "count <current_time>": Returns the number of unexpired tokens at the given *current_time*.

Note: If a token expires at a specific time and any action is performed at that same time, the token's expiration occurs before any other actions are carried out.

**Example**

Suppose *time_to_live* **= 5, and** ***queries*** **= [["generate** ***aaa 1"], ["renew aaa 2"], ["***count ***6"], ["generate bbb 7"], ["renew aaa 8"], ["renew bbb 10"], ["***count ***15"]].***

The output of the system is [1, 0]. At t = 6, the only unexpired token is "aaa". At t = 15, all tokens have expired, so the count of unexpired tokens is 0.

**Function Description**

Complete the function *getUnexpiredTokens* in the editor below.

*getUnexpiredTokens* has the following parameter(s):

*int time_to_live:* the time to live for a token

***string queries****[q]:* the queries

**Returns**

*int[]:* the results from the count query in the same order it is given.

**Constraints**

- $1 \le q \le 10000$
- $1 < |$ ***queries*** $| \le 10^5$
- $1 \le time_to_live \le 10^8$
- $1 \le current_time \le 10^8$
- $1 \le |$ *token_id* $| \le 5$
- *token_id* consists only of lowercase letters and numbers.
- All queries of type generate will contain unique values of *token_id*.
- *current_time* is in non-decreasing order in the queries.

## ▼ Input Format For Custom Testing

The first line contains an integer, *time_to_live*.
The next line contains an integer, *q*, the number of elements in *queries*.

Each line *i* of the subsequent *q* lines (where $0 \le i < q$) contains a query.

## ▼ Sample Case 0

**Sample Input For Custom Testing**

```
STDIN                       FUNCTION
-----                       --------
35                  →       time_to_live = 35
6                   →       queries[] size q = 6
generate token1 3   →       queries = ["generate token1 3", "count 4",
"generate token2 6", "count 7", "generate token3 11", "count 41"]
count 4
generate token2 6
count 7
generate token3 11
count 41
```

**Sample Output**

```
1
2
1
```

**Explanation**

| Query No. | Action | Generated Token | Timestamp | Expiry Timestamp | Unexpired Tokens |
|---|---|---|---|---|---|
| 1 | Generate | token1 | 3 | 38 | |
| 2 | Count | | 4 | | 1 |
| 3 | Generate | token2 | 6 | 41 | |
| 4 | Count | | 7 | | 2 |
| 5 | Generate | token3 | 11 | 46 | |
| 6 | Count | | 41 | | 1 |

## ▼ Sample Case 1

**Sample Input For Custom Testing**

```
STDIN                       FUNCTION
-----                       --------
9                   →       time_to_live = 9
6                   →       queries[] size q = 6
generate token1 3   →       queries = ["generate token1 3", "renew token1
5", "generate token2 7", "renew token2 8", "generate token3 9", "count
12"]
renew token1 5
generate token2 7
renew token2 8
generate token3 9
count 12
```

**Sample Output**

```
3
```

**Explanation**

| Query No. | Action | Token | Timestamp | Expiry Timestamp | Unexpired Tokens |
|---|---|---|---|---|---|
| 1 | Generate | token1 | 3 | 12 | |
| 2 | Renew | token1 | | 14 | |
| 3 | Generate | token2 | 7 | 16 | |
| 4 | Renew | token2 | | 17 | |
| 5 | Generate | token3 | 9 | 18 | |
| 6 | Count | | 12 | | 3 |

INTERVIEWER GUIDELINES

▼ **Solution**

**Skills:** e.g., Loops and counters, I/O, dynamic programming

The system supports three types of queries: "generate <token_id> <current_time>" which creates a new token with the given token_id and current_time, "renew <token_id> <current_time>" which renews an unexpired token with the given token_id and current_time (the request is ignored if there is no unexpired token with the specified token_id), and "count <current_time>" which returns the number of unexpired tokens at the given current_time.

The implementation uses a heap to store tokens sorted by their expiration times. A defaultdict is used to keep track of the frequency of each token. When a token expires, it is added to a set of expired tokens. A set is used to keep track of all tokens generated so far.

**Optimal Solution:**

```
import heapq
from collections import defaultdict

def getUnexpiredTokens(time_to_live, queries):
    heap = []
    freq = defaultdict(int)
    expired = set()
    total = set()
    ans = []

    def removeStale(current_time):
        """
        Remove expired tokens from the heap and update the expired and
total sets.
        """
        while heap and heap[0][0] <= current_time:
            freq[heap[0][1]] -= 1
            if freq[heap[0][1]] == 0:
                expired.add(heap[0][1])
            heapq.heappop(heap)

    for q in queries:
        query_type, *args = q.split()
        current_time = int(args[-1])

        if query_type == 'generate':
            token_id = args[0]
            heapq.heappush(heap, (current_time + time_to_live, token_id))
            total.add(token_id)
```

```
            removeStale(current_time)
            freq[token_id] += 1

        elif query_type == 'renew':
            token_id = args[0]
            removeStale(current_time)
            if token_id not in expired:
                heapq.heappush(heap, (current_time + time_to_live,
token_id))
                freq[token_id] += 1

        elif query_type == 'count':
            removeStale(current_time)
            ans.append(len(total) - len(expired))

        else:
            assert False

    return ans
```

**▼ Complexity Analysis**

**Time Complexity** - O(QlogN)
This is because the algorithm processes each query in O(logN) time due to the use of heap data structure. Additionally, for the 'renew' and 'generate' queries, there is an overhead of O(logN) for maintaining the heap and O(1) for updating the frequency count in the dictionary. For the 'count' queries, the algorithm just returns the number of unexpired tokens and takes O(1) time.

**Space Complexity** - O(N)
This is because we are storing each token in the heap and keeping track of the frequency count of each token in the dictionary.

**CANDIDATE ANSWER**

Language used: **TypeScript**

```typescript

/*
 * Complete the 'getUnexpiredTokens' function below.
 *
 * The function is expected to return an INTEGER_ARRAY.
 * The function accepts following parameters:
 *  1. INTEGER time_to_live
 *  2. STRING_ARRAY queries
 */

function getUnexpiredTokens(time_to_live: number, queries: string[]):
number[] {
    // Write your code here

}
```

**Result:** Compilation Failed

Compile Message

```
Solution.ts(37,71): error TS2355: A function whose
declared type is neither 'undefined', 'void', nor 'any' must return a
value.
```

No Comments